SwiftQ patient app — P2 & P3 handback

Local SQLite schema + sync outbox (P2) · Join flow with UUIDv4 idempotency (P3) · 13 Oct 2026

1. Summary

What was built. A Room/SQLite database (5 tables, schema v1, exported JSON), an encrypted sync outbox with strict write order and the Table 12 backoff, an `OutboxProcessor.flush()` ready for the P4 worker, anonymous-auth + Retrofit networking, and the join flow (select clinic → select service → priority → check and join → "You're in" → ticket status) in Jetpack Compose, English and Afrikaans. Every join writes the ticket and its outbox row in one transaction, with one UUIDv4 key that is reused by every retry together with the exact same payload bytes.

> **What works (run on the emulator, Pixel 10 Pro XL AVD, Android 16-class image, against a contract mock of the API):** online join; offline join with "place saved" then confirmation on reconnect; app killed mid-request then replayed with the same key (one ticket); paused queue then resume; triple tap → one ticket; Back never reopens Confirm; closed clinics and not-yet-open services are disabled with a reason. 25 JVM unit tests and 5 instrumented tests pass.

> **What does NOT work against the real dev API (blocker is on the API side, not the app):** the deployed dev API (`swiftq-api-dev.onrender.com`) has **no patient sign-in**, **no GET /v1/clinics**, **no /v1/sync/pushlpull** and **no GET /v1/tickets/mine**. A patient therefore cannot get a token or list clinics there, so the happy path against dev is **NOT VERIFIED**. The real-dev screenshot shows the app handling it gracefully (plain-language error + Try again). Exactly what to add is in section 8. For the emulator runs I wrote a small contract mock (`docs/tools/mock_api.py`) that copies the join endpoint's real behaviour; it is a test harness, not part of the app.

Stack note. P1 is Jetpack Compose (not XML), so all new UI is Compose and uses P1's tokens (`SwiftQTheme.colors`, `SwiftQTextStyles`, `PrimaryButton`, `SettingsChoiceTile`). The Material components and tokens in `res/` that the brief expected do not exist; the tokens live in `ui/theme`.

2. How to run and test

```
export JAVA_HOME="/Applications/Android Studio.app/Contents/jbr/Contents/Home"
export PATH=$JAVA_HOME/bin:~/Library/Android/sdk/platform-tools:~/Library/Android/sdk/emulator:$PATH
emulator -avd Pixel_10_Pro_XL &              # the only AVD on this Mac (arm64)
cd SwiftQ
./gradlew testDebugUnitTest                  # 25 JVM tests
./gradlew connectedDebugAndroidTest          # Keystore + real SQLite tests (emulator)

# Against the contract mock (what the screenshots used)
python3 docs/tools/mock_api.py 8787 &
API_BASE_URL=http://10.0.2.2:8787 FIREBASE_WEB_API_KEY=mock
FIREBASE_AUTH_BASE_URL=http://10.0.2.2:8787 \
  ./gradlew installDebug

# Against the real dev API (needs the API fixes in section 8, plus a Firebase web API key)
API_BASE_URL=https://swiftq-api-dev.onrender.com FIREBASE_WEB_API_KEY=<web key> ./gradlew installDebug
```

All endpoints and keys are `BuildConfig` fields filled from `SwiftQ/.env` or environment variables (`API_BASE_URL`, `FIREBASE_WEB_API_KEY`, `FIREBASE_AUTH_BASE_URL`, `FIREBASE_TOKEN_BASE_URL`,

`SYNC_PUSH_ENABLED`). I did not create a `.env`. Mock admin calls: `POST /__admin/queue {"queue_id":"q_general","status":"paused|open"}`, `/__admin/hold {"seconds":25}`, `/__admin/reset`, `GET /__admin/state`.

Test data needed on the real dev API: a clinic that is open now (e.g. "Olievenhoutbosch Clinic", created with `POST /v1/admin/clinics` including today's opening hours) with a service point ("General consultation", `POST /v1/admin/clinics/{id}/service-points`); the queue opens automatically on the first join. The mock uses Olievenhoutbosch Clinic (open), Laudium Community Health Centre (open), Erasmia Clinic (closed); service points General consultation (open) and Chronic medication (no queue today).

Manual test checklist (what I actually ran)

Test	Result	Evidence
Online join → ticket number within a few seconds	**Pass (mock) NOT VERIFIED on real dev API**	"You're in! A-001" in <4 s; 1 POST with Idempotency-Key
Airplane mode → join → "place saved" → reconnect → number appears, one ticket on server	**Pass**	`adb shell cmd connectivity airplane-mode enable|disable` worked. Saved screen, then A-001 about 20 s after reconnect; server held 1 ticket, saw 1 POST
Kill app while a join is in flight → relaunch → same key replayed → one ticket	**Pass**	Mock held the response 25 s, app force-stopped, relaunched. Server log: 2 POSTs, identical key and identical body; 1 ticket
Double-tap Join quickly → one ticket	**Pass**	3 simultaneous `input tap`s → 1 POST, 1 ticket (also unit + instrumented tests)
Back from status doesn't reopen Confirm	**Pass**	Back from status landed on Home
Paused queue → friendly message, joins once resumed	**Pass**	"This queue is paused… Your place is saved."; after resume the ticket appeared; 3 POSTs, same key, 1 ticket
Closed clinic and inactive/not-open service can't be selected	**Pass**	Erasmia shown greyed with reason; "Chronic medication — Not open yet today" greyed, no chevron. (Inactive service points are not returned for patients by the API, so they are filtered out.)
All targets ≥ 48 dp	**Pass**	uiautomator bounds ÷ 3.5 density on clinic list, service, priority, confirm: every app control ≥ 48 dp (only the two system status-bar icons were smaller)
TalkBack reads every control	**NOT VERIFIED**	Every control has a label or content description (inspected in the uiautomator tree); I did not run TalkBack itself. Please do one pass with it on.

3. Screens

Figma → app mapping

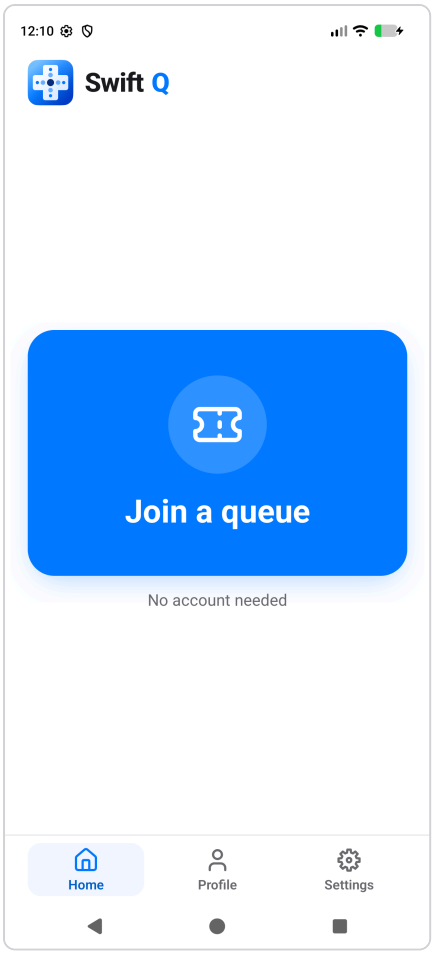
Frame	Name in Figma	App screen / state	Route	Notes
02	Home (no active ticket)	P1 `HomeScreen`; "Join a queue" now opens the flow	enum `AppScreen.Join`	If a ticket is already active the flow opens on its status instead
03	Home (active ticket)	`TicketStatusScreen` (number, people before you, wait range, queue state)	`status/{ref}`	"See live queue" (P5) and "Leave queue" (P7) deliberately not built; saved/paused/failed states added
04	Select clinic	`ChooseClinicScreen`	`join/clinic`	Search, closed/inactive disabled with reason, loading / empty / offline-with-cache / offline-no-cache / error. Distance and "N waiting" not shown: the API's Clinic has neither
—	(no frame)	`ChooseServiceScreen`	`join/service`	Added: the API's queue belongs to a service point. "Not open yet today" when `queue_id` is null
05	Priority	`PriorityRequestScreen`	`join/priority`	4 tiles + "No, I am fine". Category names appear only here because the patient must choose
06 / 06b	Confirm join (SMS / Email)	`CheckAndJoinScreen`	`join/confirm`	Alert method App / SMS / WhatsApp (the API has no email channel). Optional first name and phone added here (no separate details frame). Priority shown as a neutral icon
07	You're in	`JoinedScreen`	`joined/{ref}`	While `pending_sync` it reads "Your place is saved" instead of a number
11, 12	Confirm (signed in), Live queue	Not in scope	—	—

Tokens used

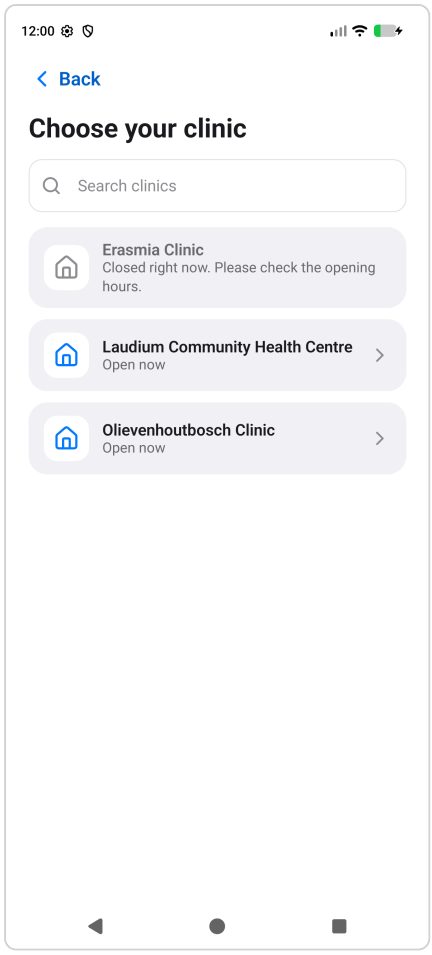
Colours (`SwiftQTheme.colors`): background, surface, grouped (cards), label, secondary, tertiary, separator, blue (primary, check mark), blueText (links, labels), blue50 (selected tile, notices). **Type** (`SwiftQTextStyles`): LargeTitle (ticket number), Title1 (screen titles), Title3, Headline, Body, Subhead, Footnote. **Components reused:** `PrimaryButton`, `SettingsChoiceTile` (priority tiles and alert method), and the Back-row style of `SettingsDetailScreen`. New shared pieces: `FlowScaffold`, `InfoCard`, `NoticeCard`, `StateMessage` (`ui/components/FlowComponents.kt`) and 8 line icons in the P1 stroke style.

> **Token gaps (for your decision).** (1) No success-green or danger-red token, so the "You're in" check uses blue and errors use Material's theme error colour. (2) P1 has no spacing/radius tokens; the new screens use the same dp literals P1 uses (24 dp gutter, 14/20 dp radii). (3) Figma's category-specific priority icons are not in the project; one neutral icon is used. (4) Only en and af exist; zu and nso are not supported by P1, so no TODO stubs were made.

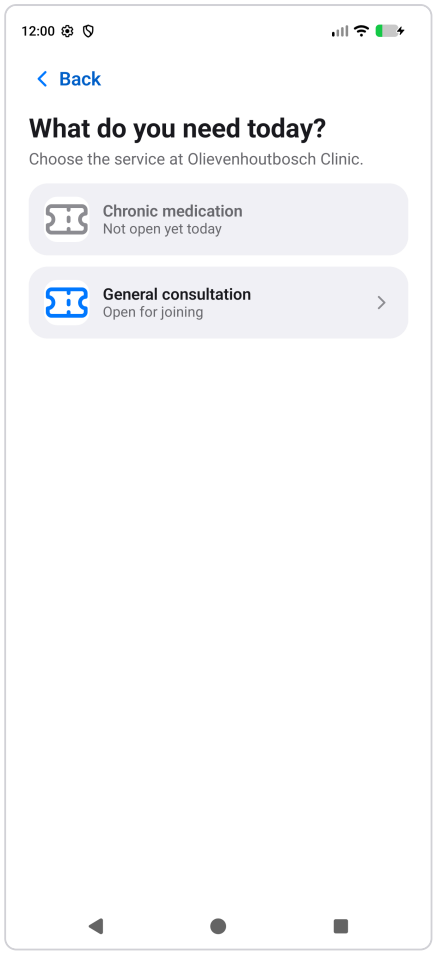
English



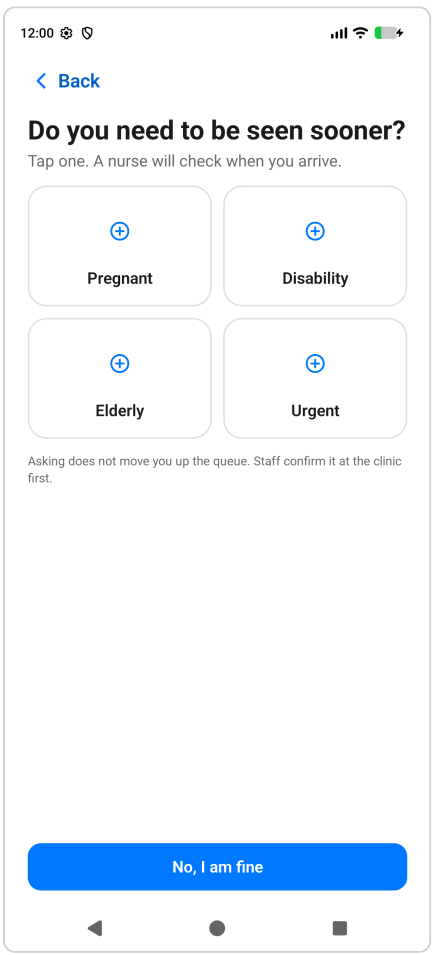
02 · Home



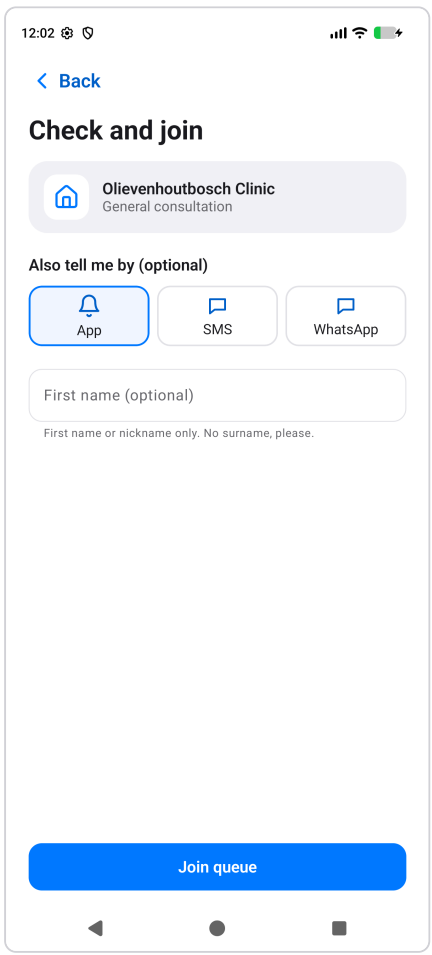
04 · Select clinic (closed clinic disabled)



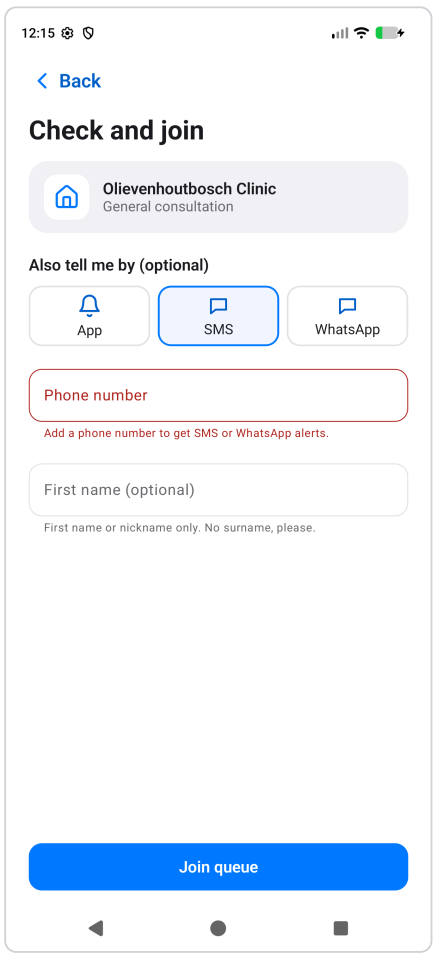
Select service ("not open yet today")



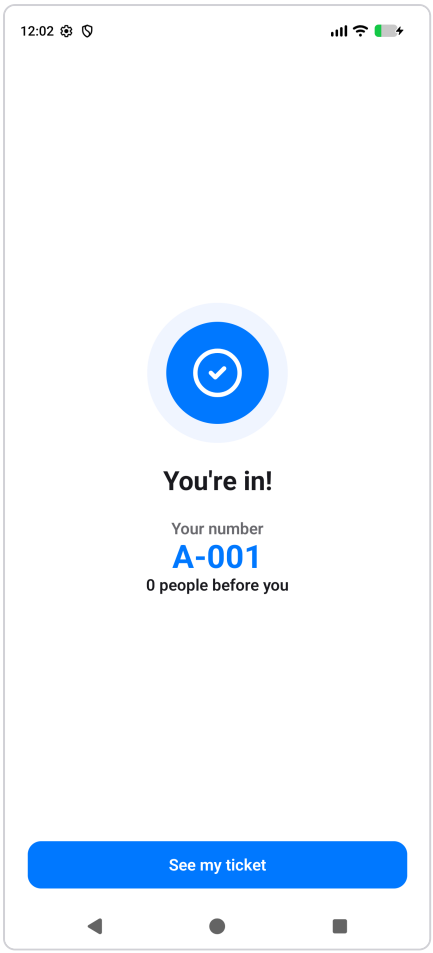
05 · Priority



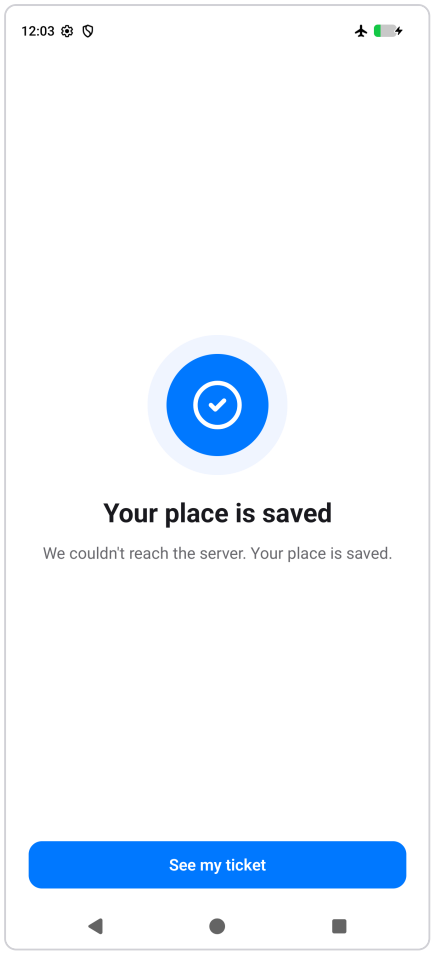
06 · Check and join



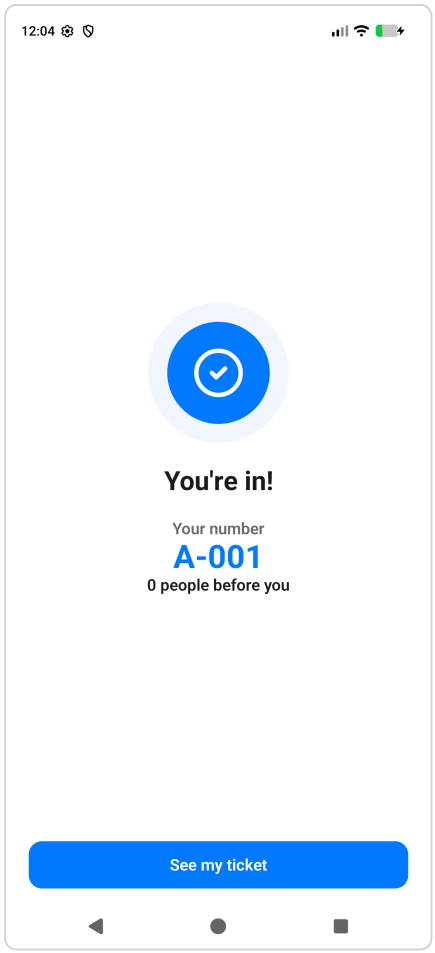
06 · inline validation (SMS needs a phone)



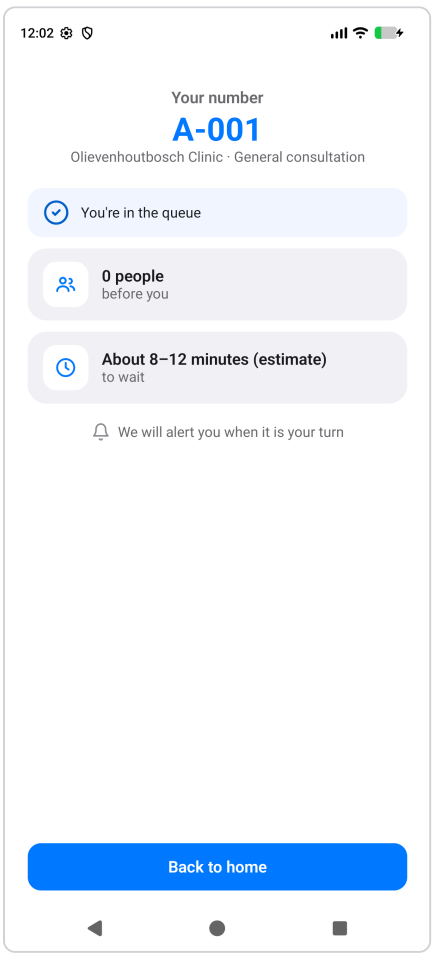
07 · You're in



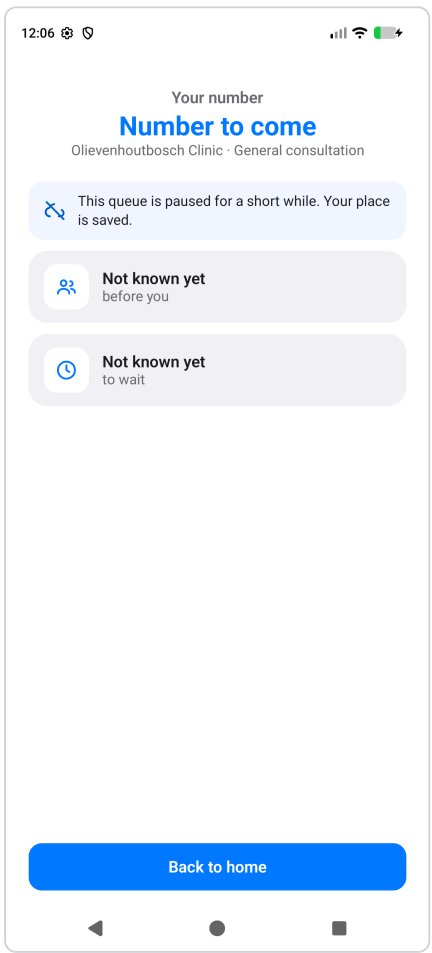
07 · offline: place saved



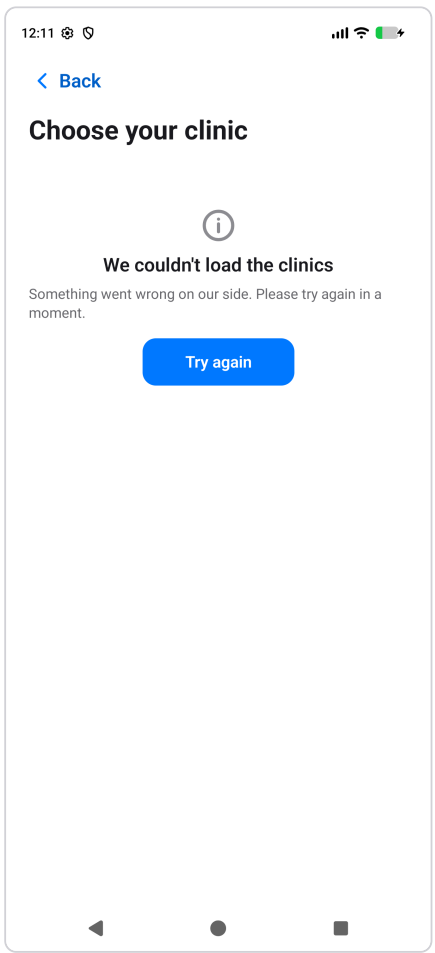
07 · after reconnect (same screen updates)



03 · Ticket status

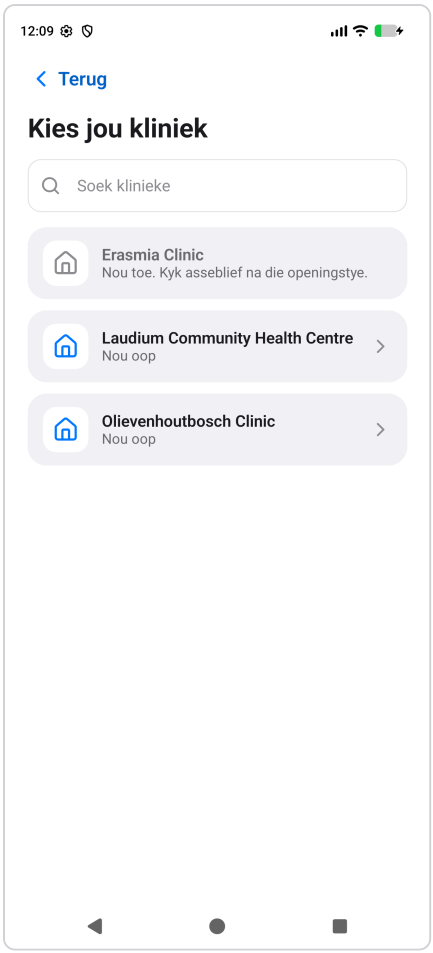


03 · queue paused, place saved

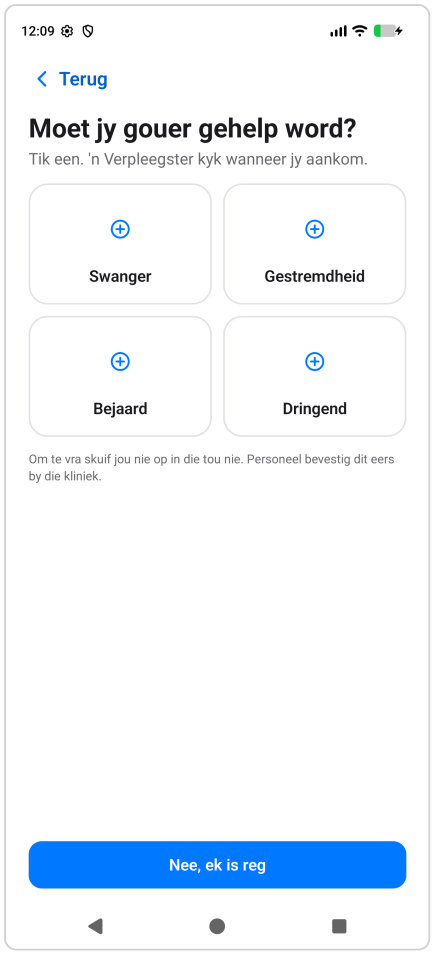


Real dev API: clinics endpoint missing

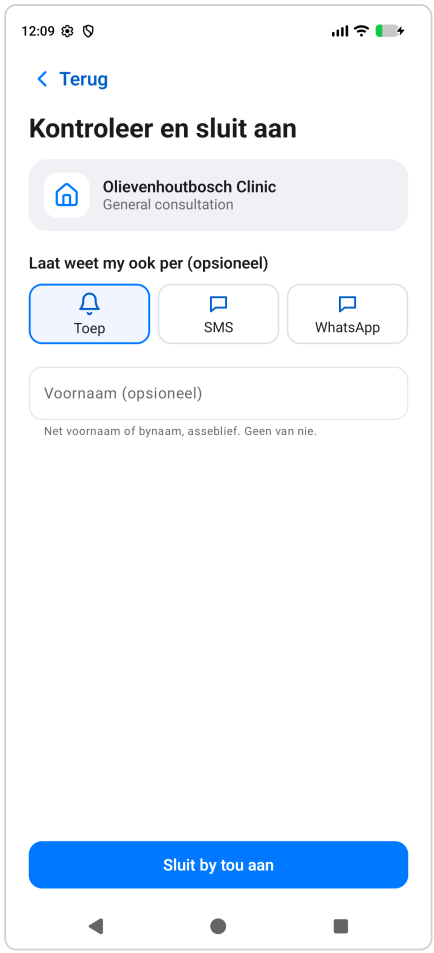
Afrikaans



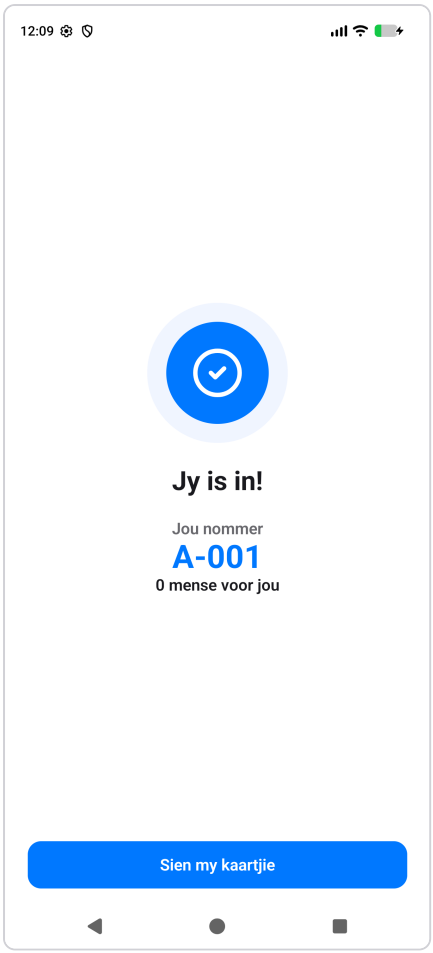
04 · Kies jou kliniek



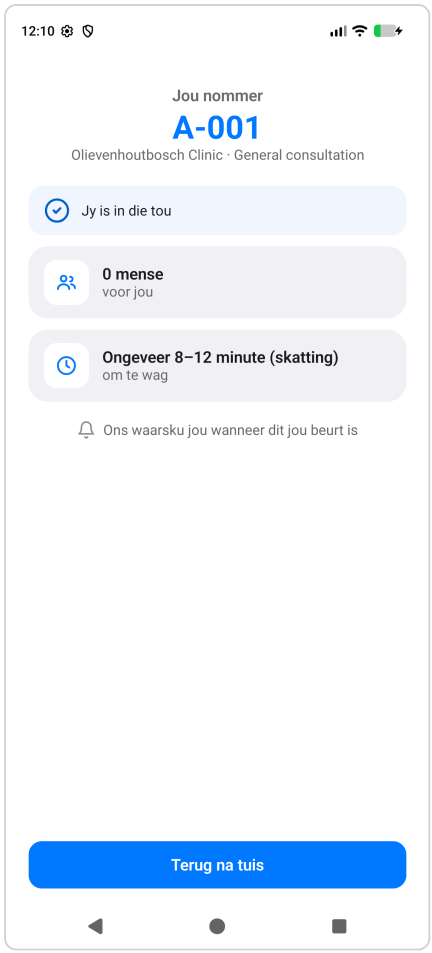
05 · Prioriteit



06 · Kontroleer en sluit aan



07 · Jy is in!



03 · Jou kaartjie

4. Database (P2) — for Chapter 4.5.1

Room 2.8 on SQLite, **version 1**, no destructive migration (the builder has no `fallbackToDestructiveMigration`). Exported schema JSON: `SwiftQ/app/schemas/com.example.swiftq.data.local.SwiftQDatabase/1.json`. Columns are snake_case like the API; booleans are INTEGER 0/1. The tables below are generated from that JSON.

clinics

column	type	null	key / default
id	TEXT	NOT NULL	PK
name	TEXT	NOT NULL	
address	TEXT	null	
is_open	INTEGER	NOT NULL	
is_active	INTEGER	NOT NULL	default 1
opening_hours_json	TEXT	null	
updated_at	TEXT	NOT NULL	
fetched_at	INTEGER	NOT NULL	

service_points

column	type	null	key / default
id	TEXT	NOT NULL	PK
clinic_id	TEXT	NOT NULL	
name	TEXT	NOT NULL	
icon	TEXT	null	
is_active	INTEGER	NOT NULL	
queue_id	TEXT	null	
concurrent_servers	INTEGER	NOT NULL	default 1
updated_at	TEXT	NOT NULL	
fetched_at	INTEGER	NOT NULL	

Indexes: index_service_points_clinic_id (clinic_id)

Foreign keys: clinic_id → clinics(id) ON DELETE CASCADE

tickets

column	type	null	key / default
local_ref	TEXT	NOT NULL	PK
server_id	TEXT	null	
queue_id	TEXT	NOT NULL	
service_point_id	TEXT	NOT NULL	
clinic_id	TEXT	NOT NULL	
display_number	TEXT	null	
sequence_no	INTEGER	null	
status	TEXT	NOT NULL	
channel	TEXT	NOT NULL	default 'app'
position	INTEGER	null	
people_ahead	INTEGER	null	
wait_min_minutes	INTEGER	null	
wait_max_minutes	INTEGER	null	
wait_cold_start	INTEGER	null	
counter_label	TEXT	null	
priority_requested	TEXT	NOT NULL	default 'none'
priority_confirmed	INTEGER	NOT NULL	default 0
late_flag	INTEGER	NOT NULL	default 0
display_name	TEXT	null	
phone_enc	TEXT	null	
phone_last4	TEXT	null	
notification_channel	TEXT	NOT NULL	
language	TEXT	NOT NULL	
client_created_at	TEXT	NOT NULL	
created_at	TEXT	null	
updated_at	TEXT	NOT NULL	
called_at	TEXT	null	
closed_at	TEXT	null	
last_synced_at	INTEGER	null	
hidden_from_history	INTEGER	NOT NULL	default 0

Indexes: index_tickets_server_id (server_id) UNIQUE; index_tickets_clinic_id_status (clinic_id, status)

outbox

column	type	null	key / default
id	INTEGER	NOT NULL	PK
idempotency_key	TEXT	NOT NULL	
op	TEXT	NOT NULL	
queue_id	TEXT	null	
ticket_id	TEXT	null	
local_ref	TEXT	NOT NULL	
payload_enc	TEXT	NOT NULL	
client_updated_at	TEXT	NOT NULL	
state	TEXT	NOT NULL	
attempt	INTEGER	NOT NULL	default 0
next_attempt_at	INTEGER	NOT NULL	
last_error_code	TEXT	null	
last_error_message	TEXT	null	
created_at	INTEGER	NOT NULL	
updated_at	INTEGER	NOT NULL	

Indexes: index_outbox_idempotency_key (idempotency_key) UNIQUE; index_outbox_state_next_attempt_at (state, next_attempt_at); index_outbox_local_ref (local_ref)

Foreign keys: local_ref → tickets(local_ref) ON DELETE CASCADE

sync_state

column	type	null	key / default
key	TEXT	NOT NULL	PK
value	TEXT	null	

Why `pending_sync` and `failed` are local-only. The server only knows waiting / called / served / no_show / cancelled. `pending_sync` means "the patient tapped Join but the server has not confirmed yet", which by definition the server cannot report. `failed` means the server refused or the 24 h window expired, so no server ticket exists. Keeping them out of the API enum means the app can show a saved place honestly without inventing a server state. `phone_enc` and `payload_enc` are AES-256-GCM with an Android Keystore key (`KeystoreCipher`); the plaintext phone is never on disk.

5. Sync outbox (P2)

```
pending ──flush picks it (due, in id order)──▶ in_flight ──server answers──▶ done
   ▲                                              │  ├─ rejected          (will never be accepted;
ticket → failed)
   │  timeout / offline / 5xx / 429 / QUEUE_PAUSED │  ├─ conflict_pending  (IT admin resolves; patient
still sees saved place)
   └──────── attempt+1, next_attempt_at per Table 12 ◀┘  └─ expired          (older than 24 h; ticket →
failed)
App start: any in_flight row is reset to pending (the process may have died mid-request).
```

Attempt	1	2	3	4	5	6+
Delay before it	immediate	5 s	15 s	45 s	135 s	1 hour, until 24 h after created_at, then **expired**

If the server sends `Retry-After` the longer of that and the schedule wins. `nextAttemptDelay(attempt)` is a pure function in `sync/BackoffPolicy.kt` with unit tests for every row. A flush is triggered after a join, on app foreground, and when a network appears; `// P4:` hooks mark where WorkManager replaces them.

6. Join flow and idempotency (P3) — for Chapter 4.4.1 / 4.4.2

```
tap "Join queue"
 └─ button disabled; JoinViewModel in-flight flag (guard 1)
     └─ JoinRepository.join()  — Mutex (guard 2)
         ├─ duplicate check: active ticket at this clinic in Room? → open that ticket, write nothing
         ├─ validate (name ≤ 40, no < >; phone → +27 form; SMS/WhatsApp need a phone)
         ├─ mint local_ref and idempotency key (UUIDv4, once)
         ├─ serialise the JSON body ONCE
         └─ ONE transaction: INSERT tickets(pending_sync) + INSERT outbox(pending, key, payload_enc)
     └─ navigate to "You're in" with popUpTo(join_graph){inclusive}   (Back cannot reach Confirm)
     └─ OutboxProcessor.flush()  → POST /v1/queues/{id}/tickets  with Idempotency-Key
            201/200 → ticket gets server id, number, position, wait · outbox done
            400 retryable (QUEUE_PAUSED) → stay pending · 400 not retryable → rejected, ticket failed,
server message shown
            429 → wait Retry-After · 5xx/timeout/offline → backoff · IDEMPOTENCY_KEY_REUSED →
rejected, generic message
```

Key lifecycle

- **Created** once, at the tap, inside the same transaction that writes the ticket and outbox rows (`UuidV4KeyFactory`, `UUID.randomUUID()`, asserted version 4 in a unit test).
- **Stored** in `outbox.idempotency_key` (UNIQUE).
- **Reused** on every retry, restart and reboot by reading that row. No interceptor or call adapter generates keys; the 401 handler replays the same request object.
- **Same bytes:** the body is serialised once, stored encrypted, and decrypted to the identical bytes for each attempt (MockWebServer test asserts equal key and equal body across a failed and a successful attempt).

Option chosen: REST first, no mixing — a variant of the brief's Option B

The brief's Option A needs retries through `/v1/sync/push` and Option B always uses it. The deployed API has no `/v1/sync/*` at all, so both would fail. I send **every attempt of a join through** `POST /v1/queues/{id}/tickets` (the endpoint that exists, with the same key every time), never mixing endpoints for one key. A `SyncPushTransport` (batch of 50, matched by key and local_ref, all five outcomes mapped) is built and unit-tested but switched off (`SYNC_PUSH_ENABLED=false`) until the API implements it. Because the API's idempotency record is per user and endpoint-agnostic only in name (see findings), staying on one endpoint per key is the safe choice.

Key code excerpts

`app/src/main/java/com/example/swiftq/data/repository/JoinRepository.kt · lines 54–59`

```
        // --- Duplicate check (T-03.2): one active ticket per patient per clinic ----------------
-----
        // "Active" = pending_sync, waiting or called. This also covers a double tap and a tap right
after a
        // restart, because the first tap's ticket is already in SQLite. The server enforces the same
rule
        // (it answers 200 with the existing ticket), but checking here means we never even queue a
second join.
        tickets.findActiveAtClinic(input.clinicId)?.let { return
JoinResult.AlreadyInQueue(it.localRef) }
```

app/src/main/java/com/example/swiftq/data/repository/JoinRepository.kt · lines 66–70

```
        // --- Identity of this intent, minted once ------------------------------------------------
        val now = clock()
        val nowIso = Instant.ofEpochMilli(now).toString()
        val localRef = keys.newKey()
        val idempotencyKey = keys.newKey() // created here and nowhere else; every retry reads it from the outbox row
```

app/src/main/java/com/example/swiftq/data/repository/JoinRepository.kt · lines 83–92

```
        // --- One transaction: the ticket and its outbox row exist together or not at all ---------
        transactor.run {
            tickets.insert(
                TicketEntity(
                    localRef = localRef,
                    queueId = input.queueId,
                    servicePointId = input.servicePointId,
                    clinicId = input.clinicId,
                    status = TicketState.PENDING_SYNC,
                    priorityRequested = input.priorityRequest,
```

app/src/main/java/com/example/swiftq/sync/OutboxProcessor.kt · lines 35–44

```
    suspend fun flush() = flushLock.withLock {
        expireOldEntries()
        val batch = outbox.nextBatch(clock(), BATCH_SIZE)
        if (batch.isEmpty()) return@withLock
        outbox.markInFlight(batch.map { it.id }, clock())
        val outcomes = transport.send(batch)
        batch.zip(outcomes).forEach { (entry, outcome) -> apply(entry, outcome) }
    }

    /** Epoch ms of the next entry that is waiting for its retry time, or null when nothing is pending. */
```

app/src/main/java/com/example/swiftq/sync/OutboxProcessor.kt · lines 52–80

```
    private suspend fun apply(entry: OutboxEntity, outcome: SendOutcome) {
        val now = clock()
        when (outcome) {
            is SendOutcome.Applied -> {
                // applied / duplicate / conflict_resolved / 200 replay / 201: the server's ticket is final.
                tickets.applyServerTicket(entry.localRef, outcome.ticket)
                outbox.update(entry.copy(state = OutboxState.DONE, attempt = entry.attempt + 1, ticketId = outcome.ticket.id,
                    lastErrorCode = null, lastErrorMessage = null, updatedAt = now))
            }
            SendOutcome.ConflictPending ->
                // The IT admin resolves it. The ticket stays pending_sync, so the patient still sees their saved place.
                outbox.update(entry.copy(state = OutboxState.CONFLICT_PENDING, attempt = entry.attempt + 1, updatedAt = now))
            is SendOutcome.Rejected -> {
                // Never retried, and never retried with a new key (that could create a second ticket).
                outbox.update(entry.copy(state = OutboxState.REJECTED, attempt = entry.attempt + 1,
                    lastErrorCode = outcome.code, lastErrorMessage = outcome.message, updatedAt = now))
                if (entry.op == OutboxOp.JOIN) tickets.markFailed(entry.localRef)
            }
            is SendOutcome.Retry -> scheduleRetry(entry, outcome, now)
            SendOutcome.NotAttempted ->
                outbox.update(entry.copy(state = OutboxState.PENDING, updatedAt = now))
        }
    }

    private suspend fun scheduleRetry(entry: OutboxEntity, retry: SendOutcome.Retry, now: Long) {
```

```
        val attempts = entry.attempt + 1
        when (val decision = BackoffPolicy.afterFailure(attempts, entry.createdAt, now,
retry.retryAfterSeconds)) {
            is RetryDecision.RetryAt -> outbox.update(entry.copy(state = OutboxState.PENDING, attempt
= attempts,
                nextAttemptAt = decision.epochMs, lastErrorCode = retry.code, lastErrorMessage =
retry.message, updatedAt = now))
```

7. Findings

Answers to the Phase 0 questions (from the API code)

- **Join request.** `JoinQueueRequest`: display_name (≤ 40, no < >), phone (`^\+27[0-9]{9}$`), language (`^[a-z]{2,3}$`, required), notification_channel (push|sms|whatsapp|none), priority_request, client_created_at. Unknown fields → 422 (`extra=forbid`). SMS/WhatsApp without a phone → 422.
- **Responses.** 201 + Location for a new ticket; 200 for an idempotent replay or when the caller already holds an active ticket. Errors use the envelope with a plain-language `message`; CLINIC_CLOSED / SERVICE_POINT_INACTIVE / QUEUE_CLOSED are 400 not retryable, QUEUE_PAUSED is 400 retryable, IDEMPOTENCY_KEY_REUSED 400, VALIDATION_FAILED 422, UNAUTHENTICATED 401.
- **Idempotency.** Header `Idempotency-Key`, parsed as a `uuid.UUID` (any valid UUID, not strictly v4). Redis key `idem:{uid}:{key}`, SET NX, TTL 24 h. The hash covers `("join", queue_id, parsed body model)`, **not the raw bytes**, so key order or whitespace would not matter; the app still resends identical bytes. The replay check runs before the duplicate check and before the paused/closed checks, so a replay of a successful join still answers 200 while paused.
- **Cross-endpoint question: not answerable by running code.** `/v1/sync/push` is not implemented, so a key used on `/queues/{id}/tickets` cannot be replayed through it. The hash is namespaced with the literal `"join"` for the REST route, so whichever way sync is implemented it must reuse `idempotency_service.request_hash("join", …)` for a shared key to be recognised. This is why the app does not mix endpoints.
- **Duplicate rule (T-03.2).** Implemented per **queue and per user id** (`owner_uid` in that queue's tickets with status waiting/called), not per clinic and not per device. It returns 200 with the existing ticket. The app's check is per clinic, which is stricter and prevents a second join at another service of the same clinic from being sent; if you want per-clinic on the server, that is an API change.
- **Auth.** `core/auth.py` accepts only the API's own HS256 JWT (`role` claim). Tokens are issued only by `/v1/auth/login` (staff email + password) and refresh. There is **no Firebase ID-token verification and no anonymous/patient sign-in** in the code, so the brief's "anonymous Firebase auth" has nothing to talk to. The app signs in to Firebase anonymously over REST (needs a web API key) and sends that ID token as Bearer; the API must be changed to accept it.
- **Sync.** `OutboxItem`, 50-item cap, five outcomes and `SyncPullResponse` exist as schemas only; no routes.

API code vs contract mismatches

- In the contract but not deployed: `GET /v1/clinics`, `/v1/clinics/{id}`, `GET /v1/tickets/mine`, `POST /v1/sync/push`, `GET /v1/sync/pull`, `/v1/service-points/{id}/queues`, queue close/config, ticket serve/recall/return-late/dismiss/transfer/notifications, `PATCH /v1/tickets/{id}`, and most admin routes.
- In the code but not the contract: `/v1/auth/login|register|refresh` (HS256 staff login).
- `GET /v1/clinics/{id}/service-points` always returns a non-null `queue_id` ("joining opens the queue"), whereas the contract says null until staff open one.
- Contract says joins are rate-limited (429); the join route has no limiter. `PUT/DELETE /v1/me/push-token` raise NotImplementedError (500).
- Contract: "one active ticket per *device* per clinic"; code: per user per queue.

- Ticket JSON contains no `device`/`queue state` field, so the status screen cannot show paused/open for a confirmed ticket without a separate GET /queues/{id} (P5).

Decisions made

Decision	Reason
Compose, not XML + ViewBinding	That is what P1 used; tokens are Kotlin objects.
Added a service-point step	The API's queue belongs to a service point; Figma has no such frame.
Name and phone on the Confirm screen	Figma 06 has the phone field there; no separate details frame exists.
Alert methods App / SMS / WhatsApp, no Email	API NotificationChannel has no email (code beats Figma).
Category names shown only on the choose screen	The patient must see what they are picking; Confirm and Status use a neutral icon, per the brief.
REST join for every attempt; sync/push built but off	Dev API lacks sync/push; one endpoint per key avoids any cross-endpoint ambiguity.
Anonymous Firebase sign-in over REST (Identity Toolkit)	No `google-services.json` in the repo; REST needs only a web key via BuildConfig. Without a key the app sends no token and the outbox waits.
Local duplicate check per clinic	Brief asks for it; stricter than the server and harmless.
DAOs as interfaces plus a `Transactor`	Lets repository and processor logic run as JVM tests with fakes; real SQLite atomicity is covered by instrumented tests.
Extra triggers (network callback, 5 s poll on the status screen, 15 s ticket refresh)	Needed so a reconnect confirms the ticket without restarting the app; all marked to be replaced by P4/P5.
Afrikaans translated, no zu/nso	P1 ships en and af only.
Display number "A014" shown as "A-014"	Matches the Figma frames.

What I changed in P1 work

- `MainActivity.kt`: added `AppScreen.Join` and wired Home's "Join a queue" (previously a no-op). Nothing removed.
- `AndroidManifest.xml`: INTERNET, ACCESS_NETWORK_STATE, `SwiftQApp`. `strings.xml` (en, af): appended strings. Gradle files: new dependencies and BuildConfig fields.
- P1's unused stub screens (`SelectClinicScreen`, `PriorityScreen`, `ConfirmJoinScreen`, `JoinSuccessScreen`, `LiveQueueScreen`) are left untouched; the new screens live in `ui/join` and `ui/status` and supersede them. Delete the stubs when you are ready.

Open questions for you

- Which API will accept patient tokens: Firebase ID tokens (as the brief says) or a new `/v1/auth/anonymous` that issues the HS256 JWT? The app only needs one change in `FirebaseAnonymousAuth` either way.
- Should the server duplicate rule be per clinic (contract wording) or per queue (current code)?
- Do you want a green/red token added to P1 for success and error states?

8. What the API needs before the dev run can be verified

1. A way for a patient to get a token (accept Firebase anonymous ID tokens with role `patient`, or add an anonymous-login route).
2. `GET /v1/clinics` (paged `{items, next_cursor, has_more}`) — the clinic list cannot be built without it.
3. Optional for this iteration: `/v1/sync/push`, `/v1/sync/pull`, `/v1/tickets/mine`.

4. Seed data: one open clinic with a service point (`POST /v1/admin/clinics`, `POST /v1/admin/clinics/{id}/service-points` as it_admin/manager).

Then run the dev command in section 2 and repeat the checklist. Nothing in the API folder was modified.

9. Test results

Suite	Result
`./gradlew testDebugUnitTest` (JVM)	**25 tests, 0 failures: BackoffPolicyTest (5: Table 12, Retry-After, 24 h expiry, v4 UUIDs, no collisions), JoinValidationTest (3: valid and invalid SA numbers, name rules), JoinFlowTest (16: one ticket + one outbox row, plaintext absent, rollback after first insert, double tap, retry reuses key and bytes via MockWebServer, backoff timing, 200 replay, merge of duplicate, paused/Retry-After, 400 non-retryable, KEY_REUSED never retried, 422, 429, in_flight reset, 24 h expiry, every sync/push outcome)**
`./gradlew connectedDebugAndroidTest` (emulator)	**5 tests, 0 failures: Keystore AES-GCM round trip returns identical bytes with fresh IV; real SQLite: one ticket + one outbox row under concurrent taps, constraint failure rolls back both rows, in_flight reset**

Emulator: flows in section 2 were driven with `adb shell input` and `uiautomator dump`; screenshots are from those runs. **Not tested on a device:** the real dev API happy path, the 401 token-refresh path and 429 (unit-tested only), the sync/push transport (unit-tested only), TalkBack, and release builds.

10. Files changed

- `pp/build.gradle.kts` — added
- `app/src/main/AndroidManifest.xml` — modified
- `app/src/main/java/com/example/swiftq/MainActivity.kt` — modified
- `app/src/main/res/values-af/strings.xml` — modified
- `app/src/main/res/values/strings.xml` — modified
- `build.gradle.kts` — modified
- `gradle/libs.versions.toml` — modified
- `app/schemas/` — added
- `app/src/androidTest/java/com/example/swiftq/DataLayerInstrumentedTest.kt` — added
- `app/src/debug/` — added
- `app/src/main/java/com/example/swiftq/AppContainer.kt` — added
- `app/src/main/java/com/example/swiftq/SwiftQApp.kt` — added
- `app/src/main/java/com/example/swiftq/data/local/` — added
- `app/src/main/java/com/example/swiftq/data/remote/` — added
- `app/src/main/java/com/example/swiftq/data/repository/` — added
- `app/src/main/java/com/example/swiftq/security/` — added
- `app/src/main/java/com/example/swiftq/sync/` — added
- `app/src/main/java/com/example/swiftq/ui/components/FlowComponents.kt` — added
- `app/src/main/java/com/example/swiftq/ui/join/` — added
- `app/src/main/java/com/example/swiftq/ui/status/` — added
- `app/src/main/res/drawable/ic_chat.xml` — added
- `app/src/main/res/drawable/ic_clock.xml` — added

- `app/src/main/res/drawable/ic_cloud_off.xml` — added
- `app/src/main/res/drawable/ic_info.xml` — added
- `app/src/main/res/drawable/ic_people.xml` — added
- `app/src/main/res/drawable/ic_priority.xml` — added
- `app/src/main/res/drawable/ic_retry.xml` — added
- `app/src/main/res/drawable/ic_search.xml` — added
- `app/src/test/java/com/example/swiftq/Fakes.kt` — added
- `app/src/test/java/com/example/swiftq/data/` — added
- `app/src/test/java/com/example/swiftq/sync/` — added
- `docs/` — worklog, screenshots, `tools/mock_api.py`, Figma reference PNGs, this PDF

Package layout: `data/local` (Room), `data/remote` (Retrofit, DTOs, ApiResult, auth), `data/repository`, `sync` (OutboxProcessor, BackoffPolicy, transports, key factory), `security` (KeystoreCipher), `ui/join`, `ui/status` — under the existing `com.example.swiftq`. No S7 shared module exists. PDF built with headless Google Chrome (no pandoc/wkhtmltopdf installed).